UNS Requirements for an Application

Oct 13, 2026 · @Fabian

There is no UNS certification: an application fits a Unified Namespace if it meets the requirements on this list. A UNS is an architecture, not a product. It is the central place where the current state of an industrial business is published in real time, organized in a semantic hierarchy.

Essential requirements

Without these seven requirements, an application cannot take part in a UNS.

#	Requirement	Why
1	MQTT client (publish and subscribe)	MQTT is the standard transport of a UNS
2	Event-driven publishing (report by exception)	A UNS runs on events, not polling
3	Configurable, hierarchical topics (ISA-95: enterprise/site/area/line/equipment/data point)	Every data point sits in its semantic place
4	Retained messages	The UNS reflects the current state
5	Standardized payload: JSON with value, timestamp, quality and unit, and/or Sparkplug B	Any consumer can interpret the data
6	Security: TLS, authentication (username/password or X.509 certificates), support for per-topic ACLs	The UNS is the company's data core
7	State and outage detection (Last Will or Sparkplug birth/death)	Stale data is never shown as valid

Highly recommended requirements

These eight requirements separate a basic integration from one ready for production at scale.

#	Requirement	Why
8	Native Sparkplug B	State management, typing and auto-discovery in the OT layer
9	Templates or asset models	Scale to hundreds of assets consistently
10	Bidirectional (subscribe and act, e.g. write to PLCs)	Commands and setpoints from the UNS, always under control
11	Store & Forward	No data loss if the broker connection drops
12	Configurable QoS (0/1/2)	Balance between performance and delivery guarantee
13	Failover or multiple brokers	High availability and multi-site architectures
14	MQTT 5.0	Better error handling, message expiry and metadata
15	Edge logic or scripting	Compute KPIs and normalize data close to the source

Requirements by role in the UNS

The key requirements depend on whether the application produces, distributes or consumes data.

Role	Examples	Key requirements
Edge producer	vNode, Kepware, Litmus, Ignition Edge	OT drivers, hierarchical publishing, Sparkplug B/JSON, Store & Forward, templates
Broker (hub)	HiveMQ, EMQX, Mosquitto	High availability or clustering, retained messages, ACLs, TLS, Sparkplug-aware if Sparkplug B is used

Role	Examples	Key requirements
Central integration (UNS to IT)	vNode in a centralized deployment	Subscribe and publish on the broker, and deliver data to historians, relational databases, MES, ERP, SAP and clouds such as AWS or Azure. Relay messages over other protocols: REST, OPC UA, Modbus, DNP3, IEC 104 and files
Consumer / processor	MES, SCADA, BI, AI, Node-RED, historian, vNode	Wildcard subscriptions, payload interpretation, respect for data quality, republishing results to the UNS

vNode can cover every role except the broker. At the edge it connects to OT equipment and publishes to the UNS. In a centralized deployment it subscribes with wildcards, interprets the payload, respects data quality and republishes its results to the UNS. It also feeds IT applications and relays messages over other protocols (REST, OPC UA, Modbus, DNP3, IEC 104 and files), with no separate integration to build for each system.

A good UNS application does not only consume: it also gives value back to the UNS. For example, the MES publishes the current order and the OEE system publishes its KPIs.

Architecture requirements

No application meets these on its own, but without them there is no UNS.

- **Central broker**, independent and highly available.
- **Data model and naming convention**, defined and documented.
- **Governance**: who publishes what, on which branches, and who may write.
- **Historian** subscribed to the UNS, because the UNS holds current state, not history.

One-sentence summary

An application fits a UNS if it publishes and/or consumes events over MQTT, in a semantic hierarchy, with standardized payloads, current state (retained), outage detection and security. A complete UNS also needs a central broker and a governed data model.

Applied to vNode, Node-RED and Matrikon

vNode meets every essential requirement and almost every recommended one. Node-RED can meet them if they are built. Matrikon is yet to be verified.

Application	Fit	Detail
vNode	Complete edge component	Sparkplug B, retained JSON, ISA-95 templates, bidirectional, Store & Forward and JavaScript (confirmed by the user). It has no built-in broker, which is not its role
Node-RED	Possible, with in-house development	Store & Forward, templates and Sparkplug B must be built or covered with community nodes
Matrikon	To be verified	If it only offers OPC UA, it is a source for the UNS and needs a gateway to MQTT

The vNode capabilities come from its public information and from what the user confirmed. They are worth validating with a test on real equipment and tag volumes.

Why UNS matters to integrators and end customers

A UNS turns custom, fragile and expensive integration into reusable data infrastructure. Each party gains different things.

For the end customer

- **Lower total cost of ownership**: each system connects to the UNS once, and an integration becomes a topic subscription. Changing one system does not break the others.
- **Vendor independence**: data lives in an open layer (MQTT, JSON, Sparkplug B), not inside one vendor's SCADA or MES. A system can be replaced without redoing integrations.
- **Data ready for analytics and AI**: it arrives structured (ISA-95), with context and in real time. A predictive maintenance or OEE pilot starts in days, not months.
- **Real-time decisions**: production, maintenance, quality and management see the same truth at the same time and react within seconds.
- **Multi-site scalability**: with the same structure and templates, replicating a solution at another plant is close to copy and paste.

- **Security and governance**: a single controlled access point (TLS and ACLs), with no inbound ports on the OT network. It eases alignment with IEC 62443.

For the integrator

- **Faster, repeatable projects**: asset templates, topic conventions and reference stacks reused from one customer to the next. Fewer hours per project and better margins.
- **Lower technical risk**: fewer custom integrations, fewer points of failure and less after-sales support. Commissioning can be phased.
- **Commercial differentiation**: customers increasingly ask about UNS, Industry 4.0 and AI. The integrator becomes a digital transformation partner, with multi-year roadmaps.
- **New business lines**: architecture and governance consulting, recurring services and new use cases (OEE, energy, quality, AI) on the same foundation.
- **Open ecosystem**: the best product is chosen for each layer without lock-in to one vendor, and integration with the customer's other suppliers is easy.

Side-by-side summary

Area	End customer	Integrator
Economic	Lower integration and maintenance costs	Faster projects with better margins
Strategic	Vendor independence and multi-site scalability	Positioning as a digital transformation partner
Technical	Data ready for AI and real time	Standardized, reusable solutions
Risk	Less dependency and more security	Fewer fragile integrations and less support
Growth	New use cases without rebuilding the foundation	Recurring services and account expansion

For the customer, a UNS turns industrial data into an owned, open and scalable asset. For the integrator, it turns integration into a repeatable methodology that cuts costs and opens recurring business. These benefits depend on a good data model and clear governance.